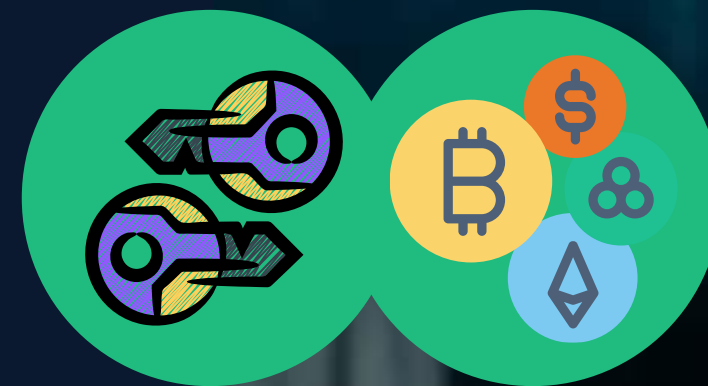


MULTIMEDIA UNIVERSITY, MALAYSIA

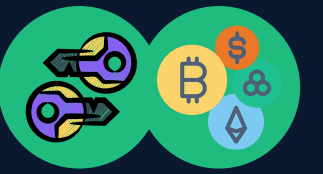
Cryptography Fundamentals

ETHICAL HACKING AND PENETRATION TESTING
CCS6324 - 2530

Dr. Hafiz Adnan Hussain

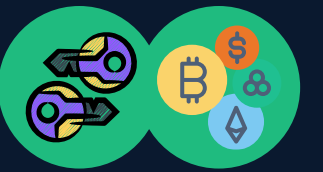


DISCLAIMER



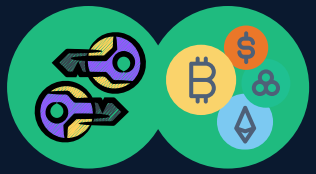
- This course is designed to educate students on ethical hacking principles. The knowledge and techniques acquired must be used responsibly and not for any illegal or unauthorized activities.
- Each student is solely responsible for their actions.
- Do not investigate individuals, websites, servers or conduct any illegal activities on any system you do not have permission to analyze.
- The course content has been developed in good faith, drawing from the author's experience and reputable sources. While every effort has been made to ensure accuracy, the author does not guarantee absolute correctness. Students are encouraged to exercise discretion and actively engage in discussions by raising any questions or concerns during the course.

NOT a Cryptography Course



- We won't cover the entire cryptography course in a single chapter.
- Focus on understanding what could go wrong.

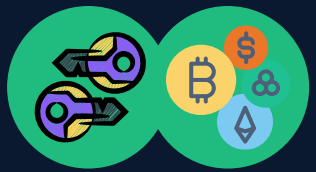
Learning Objectives



What You Will Learn?

- Define cryptography and its core principles
- Understand the evolution from classical to modern encryption
- Identify the weaknesses in historical ciphers
- Recognize cryptography's role in cybersecurity

1: Introduction to Cryptography & Historical Context



What is Cryptography?

Cryptography is the science of securing data through encoding and transformation. It converts plaintext (readable information) into ciphertext (unreadable, encrypted data), making it inaccessible to unauthorized users.

Key Definitions:

- **Plaintext:** Original readable message
- **Ciphertext:** Encrypted, unreadable message
- **Encryption:** Process of converting plaintext to ciphertext
- **Decryption:** Process of converting ciphertext back to plaintext
- **Key:** Secret parameter controlling encryption/decryption

Cryptography Goals (CIA Triad Extended)

1. **Confidentiality:** Ensure only authorized parties can read data
2. **Integrity:** Verify data hasn't been altered
3. **Authenticity:** Confirm the sender's identity
4. **Non-repudiation:** The sender cannot deny sending the message

Historical Ciphers

Substitution Cipher

The oldest known encryption method, replacing one letter with another based on a key.

Caesar Cipher Example:

- Key value of 3 shifts each letter 3 positions forward
- ABCDEFGHIJKLMNOPQRSTUVWXYZ → DEF...GHIJKLMNOPQRSTUVWXYZABC
- "HELLO" becomes "KHOOR"
- Vulnerable to frequency analysis

ROT-13

- Special case of Caesar cipher with key value of 13
- Applying ROT-13 twice returns the original text
- Example: HELLO → URYYB → HELLO
- Historically used to hide spoilers and mild content online
- Completely broken by modern standards

Enigma Machine (WWII)

- German encryption device using mechanical rotors
- Complex substitution based on rotor position and electrical pathways
- Believed unbreakable at the time
- Cracked by Alan Turing's team at Bletchley Park
- Lesson: Complexity doesn't guarantee security

Why These Failed

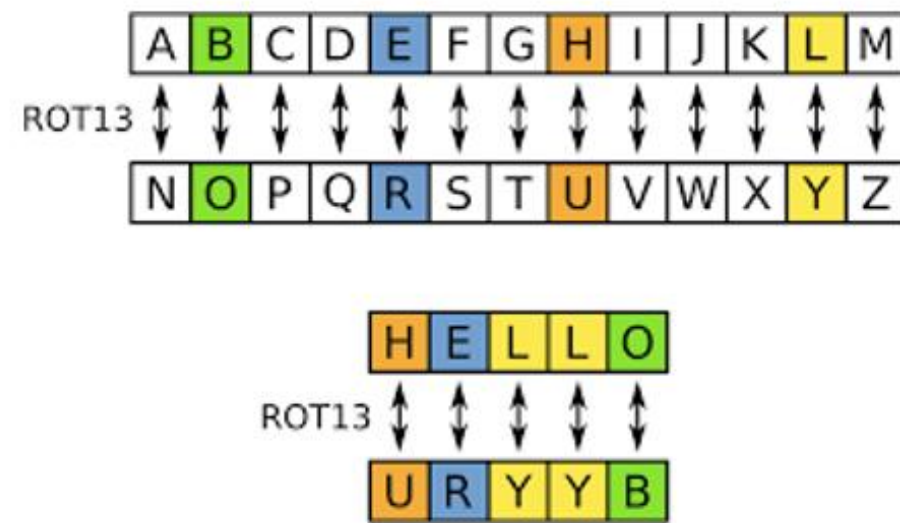
1. Small keyspace: Limited possible keys
2. Frequency analysis: Letter distribution patterns reveal structure
3. Known plaintext attacks: Patterns in messages expose the cipher
4. Lack of mathematical rigor: Designed by intuition, not proven secure

Steganography vs. Cryptography

- Steganography: Hiding information in plain sight (images, text, video)
- Cryptography: Making information unreadable
- Different goals: steganography hides existence; cryptography denies access
- Often combined for defense in depth

1: Introduction to Cryptography & Historical Context (Cont'd)

ROT-13



ROT13 replaces each letter by its partner 13 characters further along the alphabet. For example, HELLO becomes URYyb (or, reversing, URYyb becomes HELLO again)

Source: Wikipedia

Enigma



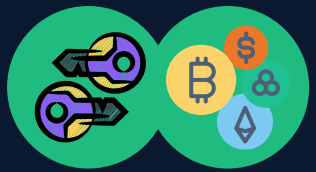
- Used by the Germans during World War II
 - Replaced letters as they were typed
 - Substitutions were computed using a key and a set of switches or rotors
 - Image from Wikipedia

Steganography



- The process of hiding data in plain view in pictures, graphics, or text
 - Example: changing colors slightly to encode individual bits in an image
- The image on the left contains the image on the right hidden in it

2: Cryptographic Foundations & Key Concepts



Algorithms and Keys

Algorithm: A mathematical function or procedure that transforms plaintext using a key.

Security depends on two factors:

1. **Strong Algorithm:** Cannot be reversed without the key
2. **Secure Key:** Cannot be found through guessing or brute force

Key Fundamentals

Key: A sequence of random bits controlling the encryption process.

Keyspace: The range of all possible key values.

- 8-bit key: $2^8 = 256$ possible values
- 56-bit key: $2^{56} = 7.2 \times 10^{16}$ possible values
- 128-bit key: $2^{128} = 3.4 \times 10^{38}$ possible values
- 256-bit key: $2^{256} = 1.2 \times 10^{77}$ possible values

Key Length Impact:

- Larger keyspace = More computational effort required to break
- Moore's Law: Computing power doubles every 2 years (key sizes must increase accordingly)
- Modern minimum: 128 bits for symmetric, 2048 bits for RSA

Brute Force Attacks

Definition: Attempting every possible key combination until the correct plaintext is found.

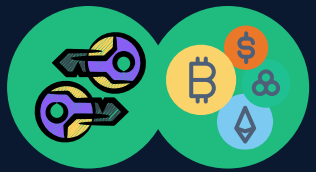
Historical Example (1997):

- 56-bit DES key broken through coordinated brute force
- Used 14,000 machines organized via the Internet
- Required 3 months of continuous work
- Today: Same attack takes minutes with modern hardware

Defense Strategy:

- Use large key sizes (128+ bits for symmetric)
- Implement rate limiting on password attempts
- Use key stretching algorithms (PBKDF2, bcrypt, Argon2)

3: Symmetric Cryptography Algorithms



Overview

Symmetric cryptography uses a single shared key for both encryption and decryption. Both parties must securely share this key.

Plaintext + Key → [Encryption] → Ciphertext

Ciphertext + Key → [Decryption] → Plaintext

■ Cleartext with Key makes Ciphertext

Winning Lotto #s:



aWDHOP#@-w9

■ Ciphertext with Key makes Cleartext

aWDHOP#@-w9



Winning Lotto #s:

Advantages of Symmetric Cryptography

- **Speed:** 100-1000x faster than asymmetric algorithms
- **Simplicity:** Straightforward mathematical operations
- **Proven Security:** Well-studied algorithms with long track records
- **Efficiency:** Uses minimal computing resources

Disadvantages of Symmetric Cryptography

- **Key Distribution Problem:** Both parties must securely share the same key
- **Scalability Issues:** n users need $n(n-1)/2$ unique keys for pairwise communication
- **No Authentication:** Cannot verify who encrypted the message
- **No Non-repudiation:** Sender can deny sending a message (both have same key)

Two Types of Symmetric Ciphers

1. Stream Ciphers

- Encrypt plaintext one bit at a time
- Use pseudo-random number generators
- Fast but more vulnerable to certain attacks
- Example: RC4

2. Block Ciphers

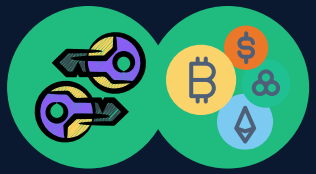
- Encrypt plaintext in fixed-size blocks (typically 64 or 128 bits)
- More secure than stream ciphers
- Modern standard approach
- Examples: DES, 3DES, AES

Symmetric Algorithms: Details & Security Status

Data Encryption Standard (DES)

- Standardized: 1977 by NIST
- Key Size: 56 bits (reduced from IBM's original 128-bit Lucifer)
- Block Size: 64 bits
- Status: **✗ INSECURE** - Do not use
- History: Broken in 1997 (3 months with 14,000 machines); broken in 3 days in 1998
- Why It Failed: 56-bit key too small; modern computers crack it in minutes

3: Symmetric Cryptography Algorithms (Cont'd)



Symmetric Algorithms: Details & Security Status (Cont'd)

Triple DES (3DES)

- Process: Applies DES encryption three times with two or three keys
- Key Size: 112 or 168 bits (depending on variant)
- Block Size: 64 bits
- Strength: 2^{56} times stronger than DES
- Status:  DEPRECATED - Being phased out
- Downside: Slow execution; requires three rounds of DES
- Current Use: Legacy banking systems, payment card industry (Phase-out by 2024)

Advanced Encryption Standard (AES)

- Standardized: 2002 by NIST (5-year development process)
- Algorithm: Rijndael (designed by Belgian cryptographers)
- Key Sizes: 128, 192, or 256 bits
- Block Size: 128 bits
- Status:  SECURE - Current industry standard
- Speed: Fast on both software and hardware
- Applications: HTTPS, VPNs, file encryption, cloud storage
- Rounds: 10 (128-bit key), 12 (192-bit), 14 (256-bit)

IDEA (International Data Encryption Algorithm)

- Key Size: 128 bits
- Block Size: 64 bits
- Status:  SECURE - But rarely used
- History: Developed 1991 by Lai and Massey
- Use: Early PGP implementations, academic research
- Limitation: 64-bit block size vulnerable to birthday attack with large volumes

Blowfish

- Key Size: 32 to 448 bits (highly flexible)
- Block Size: 64 bits
- Status:  SECURE - For legacy systems
- Creator: Bruce Schneier (renowned cryptographer)
- Use: File encryption, password hashing (bcrypt variant)
- Limitation: 64-bit block size; being replaced by AES
- Advantage: Free and public domain

RC4 (Rivest Cipher 4)

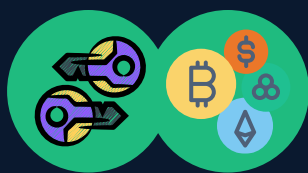
- Type: Stream cipher (most widely used before 2015)
- Key Size: 40 to 2048 bits
- Status:  BROKEN - Do not use
- History: Biases in key scheduling discovered; even creator Ron Rivest acknowledged flaws
- Previous Use: WEP, WPA (TKIP), SSL/TLS versions, HTTPS
- Vulnerabilities: Predictable key streams; can decrypt HTTPS cookies in 75 hours
- Lesson: Inventor's later warnings often ignored in standards

RC5

- Block Size: Variable (32, 64, or 128 bits)
- Key Size: Up to 2048 bits
- Status:  PARTIALLY BROKEN
- Challenge: RC5-56/64 cracked; RC5-72 project ongoing (estimated 1000 years at current rate)
- Use: Academic research, limited embedded systems

Creator: Ronald Rivest (RSA co-founder)

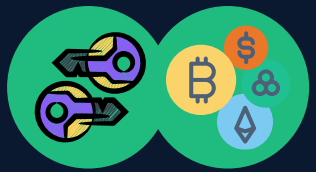
3: Symmetric Cryptography Algorithms (Cont'd)



Symmetric Algorithms Comparison Table

Algorithm	Key Size	Block Size	Speed	Status	Best For
DES	56 bits	64 bits	Moderate	✗ Insecure	None - Historical only
3DES	112-168 bits	64 bits	Slow	⚠ Deprecated	Legacy financial systems
AES	128-256 bits	128 bits	Fast	☑ Secure	Modern encryption (recommended)
IDEA	128 bits	64 bits	Moderate	☑ Secure	Legacy PGP
Blowfish	32-448 bits	64 bits	Fast	☑ Secure	Password hashing
RC4	40-2048 bits	Stream	Very Fast	✗ Broken	None - Do not use
RC5	Variable	Variable	Fast	⚠ Weakening	Research only

4: Asymmetric Cryptography (Public Key Cryptography)



Overview

Asymmetric cryptography uses **two mathematically related keys**: a **public key** and a **private key**. Data encrypted with one key can only be decrypted with the other.

Plaintext + Public Key → [Encryption] → Ciphertext

Ciphertext + Private Key → [Decryption] → Plaintext

Key Concepts

Public Key:

- Shared openly with everyone
- Used by others to encrypt messages to you
- Used to verify your digital signatures
- Cannot be used to decrypt messages meant for you

Private Key:

- Kept secret by the owner
- Used to decrypt messages sent to you
- Used to create digital signatures proving your identity
- If compromised, security is completely broken

Advantages of Asymmetric Cryptography

- **Key Distribution:** No need to securely share keys in advance
- **Scalability:** One key pair per person; millions can communicate securely
- **Authentication:** Private key proves identity through digital signatures
- **Non-repudiation:** Owner cannot deny creating a signature (proven with their private key)
- **Network-Friendly:** Solves the key exchange problem of symmetric cryptography

Disadvantages of Asymmetric Cryptography

- **Speed:** 100-1000x slower than symmetric algorithms
- **Computational Overhead:** Requires complex mathematical operations
- **Key Size:** Must be much larger (2048+ bits) for equivalent security
- **Implementation Complexity:** Easier to implement incorrectly

Hybrid Approach (Modern Standard)

Modern systems combine both:

1. **Asymmetric cryptography** for secure key exchange and authentication
 2. **Symmetric cryptography** for fast bulk data encryption
- Example: HTTPS/TLS uses RSA for key exchange, then AES for data encryption

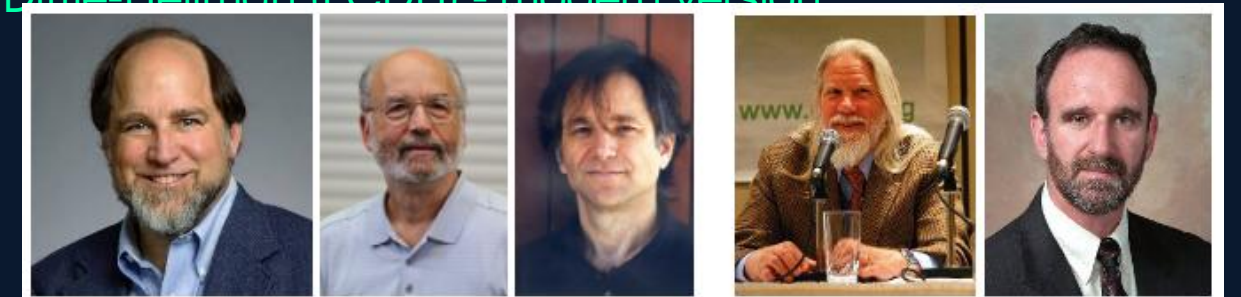
Major Asymmetric Algorithms

RSA (Rivest-Shamir-Adleman)

- **Developed:** 1977 by Rivest, Shamir, Adleman
- **Basis:** Difficulty of factoring large numbers into prime factors
- **Key Size:** 2048 bits (minimum); 4096 bits (recommended for long-term)
- **Status:** ☒ SECURE - Remains gold standard
- **Applications:** SSL/TLS certificates, HTTPS, digital signatures, email encryption
- **Security:** Unbroken for 45+ years; assumes no quantum computers exist
- **Performance:** Slower than ECC for equivalent security

Diffie-Hellman (DH)

- **Developed:** 1976 by Whitfield Diffie and Martin Hellman
- **Purpose:** Secure key exchange (NOT encryption)
- **Innovation:** Two parties agree on a secret key without ever transmitting it
- **Method:** Exchange public numbers; combine with private numbers to derive a shared secret
- **Protection:** Prevents sniffing attacks on key negotiation
- **Uses:** TLS handshakes, SSH key exchange, VPN protocols
- **Variants:** Elliptic Curve Diffie-Hellman (ECDH) - modern version



4: Asymmetric Cryptography (Public Key Cryptography) (Cont'd)

Major Asymmetric Algorithms (Cont'd)

Elliptic Curve Cryptosystems (ECC)

- **Basis:** Elliptic curve mathematical properties
- **Key Size:** 256 bits ECC $\approx$ 3072 bits RSA in security strength
- **Status:** ☒ SECURE - Modern replacement for RSA
- **Advantages:** Much smaller keys; faster computation; requires less bandwidth
- **Applications:** Mobile devices, IoT, cryptocurrencies (Bitcoin uses secp256k1)
- **Uses:** ECDSA (signatures), ECDH (key exchange), encryption
- **Trend:** Increasingly replacing RSA in new systems

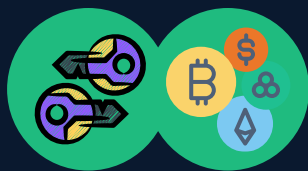
ElGamal

- **Developed:** 1985 by Taher ElGamal
- **Basis:** Discrete logarithm problem
- **Uses:** Data encryption, digital signatures, key exchange
- **Status:** ☒ SECURE - But less commonly used than RSA/ECC
- **Security:** Solving discrete logarithm takes years of CPU-intensive operations
- **Applications:** GPG (GNU Privacy Guard), PGP variants
- **Limitation:** Larger ciphertext than RSA for equivalent security

Asymmetric Algorithms Comparison

Algorithm	Key Size	Type	Status	Best For
RSA	2048-4096 bits	Encryption + Signatures	☒ Secure	General purpose, certificates
Diffie-Hellman	2048-4096 bits	Key Exchange	☒ Secure	Session key negotiation
ECC	256-521 bits	Encryption + Signatures	☒ Secure	Modern systems, mobile, IoT
ElGamal	Variable (2048+)	Encryption + Signatures	☒ Secure	PGP/GPG implementations

5: Hashing Algorithms & Data Integrity



What is Hashing?

Hashing takes a variable-length message and produces a fixed-length value called a message digest or hash.

Properties of a Good Hash Function:

- 1. Deterministic: Same input always produces same output
- 2. Fast Computation: Computing hash is quick (milliseconds)
- 3. Pre-image Resistance: Cannot reverse hash to find original input (one-way)
- 4. Small Change = Big Difference: Changing 1 bit in input dramatically changes hash (avalanche effect)
- 5. Collision-Free: No two different inputs should produce same hash
- 6. Fixed Output: All hashes same length regardless of input size

Hash as a "Digital Fingerprint"

A hash value uniquely identifies a message. If the message is modified even slightly, the hash changes completely.

Example:

Hash("The quick brown fox") = a1b2c3d4e5f6...

Hash("The quick brown dog") = z9y8x7w6v5u4...

Hash Functions vs. Encryption

Aspect	Hashing	Encryption
Purpose	Verify integrity	Hide content
Reversibility	One-way (irreversible)	Two-way (reversible with key)
Key Required	No	Yes
Output Length	Fixed	Variable
Use Case	Integrity verification, passwords	Confidentiality

Major Hash Algorithms

MD5 (Message Digest 5)

- Output Size: 128 bits (32 hexadecimal characters)
- Status: **✗** BROKEN - Collision attacks practical since 2004
- History: Created 1992 by Ronald Rivest; widely used for 15+ years
- Collision: Two different files producing same MD5 hash found 2004-2005
- Current Use: Legacy systems only (Git, file verification)
- Lesson: "Widely used" ≠ "Secure"

SHA-1 (Secure Hash Algorithm 1)

- Output Size: 160 bits (40 hexadecimal characters)
- Status: **✗** BROKEN - Collision attacks discovered 2005; practical attacks by 2017
- History: NIST standard since 1995; used in TLS, SSL, PGP, SSH, S/MIME
- Collision: SHattered attack demonstrated collisions in 2017
- Deprecation: NIST recommends discontinuing SHA-1 (officially deprecated 2020)
- Timeline: Still used in many legacy systems but being phased out

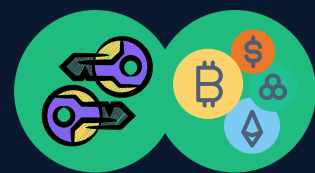
SHA-2 Family

- Variants: SHA-224, SHA-256, SHA-384, SHA-512
- Output Sizes: 224, 256, 384, 512 bits respectively
- Status: **☑** SECURE - No practical attacks known
- Recommendation: SHA-256 for new applications
- Applications: HTTPS, blockchain (Bitcoin uses SHA-256), password hashing
- Timeline: Standard since 2001; still recommended

SHA-3 (Keccak)

- Announced: 2012; finalized 2015
- Output Sizes: 224, 256, 384, 512 bits
- Status: **☑** SECURE - Designed to replace SHA-2 if needed
- Design: Completely different from SHA-1/SHA-2 (sponge construction)
- Purpose: Contingency if SHA-2 broken; also provides a performance alternative
- Current Use: Limited adoption; SHA-2 still dominant
- Future: May become primary as migration from SHA-2 happens

5: Hashing Algorithms & Data Integrity (Cont'd)



Collision Attacks

A **collision** occurs when two different inputs produce the same hash.

Birthday Attack Principle: With only 23 people in a room, there's 51% chance that two share a birthday.

23 people × 365 days = Much less than 365² combinations needed

For hashing: Finding a collision requires approximately $\sqrt{(2^n)}$ attempts, where n is the hash size in bits.

Example: For 160-bit SHA-1, a collision requires $\sim 2^{80}$ computations (vs. 2^{160} if truly random).

Practical Impact:

- MD5 (128-bit): Collision in hours with regular computers
- SHA-1 (160-bit): Collision in weeks with specialized hardware
- SHA-256 (256-bit): Collision still computationally infeasible

Hash Applications

1- Data Integrity Verification:

- Download files with hash verification (ensures no corruption)
- Version control systems (Git uses SHA-1)

2- Password Storage:

- Store the hash of the password, not the password itself
- Compare user input hash to stored hash
- If the database is compromised, passwords remain protected

3- Digital Signatures:

- Sign the hash of the document (not the entire document)
- Much faster than signing the entire document
- Proves document hasn't been altered

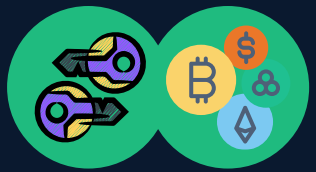
4- Malware Detection:

- Hash known malware samples
- Compare file hashes to malware databases

Hash Algorithm Summary

Algorithm	Output	Status	Best For
MD5	128 bits	✗ Broken	None - Legacy only
SHA-1	160 bits	✗ Broken	None - Being phased out
SHA-256	256 bits	☑ Secure	Modern password hashing, integrity
SHA-512	512 bits	☑ Secure	High-security applications
SHA-3	256-512 bits	☑ Secure	Future standard, contingency

6: Digital Signatures & Authentication



What is a Digital Signature?

A digital signature is a cryptographic mechanism proving:

- 1. Authentication:** Confirms the sender's identity
- 2. Integrity:** Proves the message hasn't been altered
- 3. Non-repudiation:** The sender cannot deny creating the signature

How Digital Signatures Work

Signing Process:

Document → [Hash Function] → Hash Value → [Private Key Encryption] → Digital Signature

Verification Process:

Received Signature → [Public Key Decryption] → Hash Value (received)

Received Document → [Hash Function] → Hash Value (calculated)

Compare: If equal → Valid signature; if different → Tampering detected

Advantages Over Written Signatures

- Cryptographically Verified: Mathematics proves authenticity
- Tamper-Proof: Any modification is detected immediately
- Cannot Be Forged: Requires a private key that only the owner possesses
- Legally Binding: Many jurisdictions recognize digital signatures (e-SIGN Act in US)

Digital Signature Standard (DSS)

Established: 1991 by NIST

Federal Government Requirements:

- All digital signatures use RSA or DSA (Digital Signature Algorithm)
- All signatures use a hashing algorithm for integrity
- NIST mandates the SHA family for hashing

Algorithms Approved by NIST:

- RSA: Industry standard, widely supported
- DSA: Government standard, less common commercially
- ECDSA: Elliptic Curve variant, increasingly popular

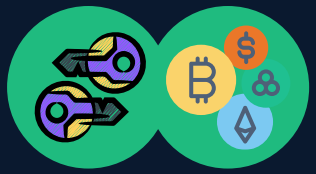
Signature Verification Process

Step-by-Step:

1. Recipient receives document + digital signature
2. Recipient decrypts the signature using the sender's public key
3. Calculate the hash of the received document
4. Compare the decrypted hash with the calculated hash
5. If match → Document authentic and unmodified
6. If no match → Either signature forged or document tampered

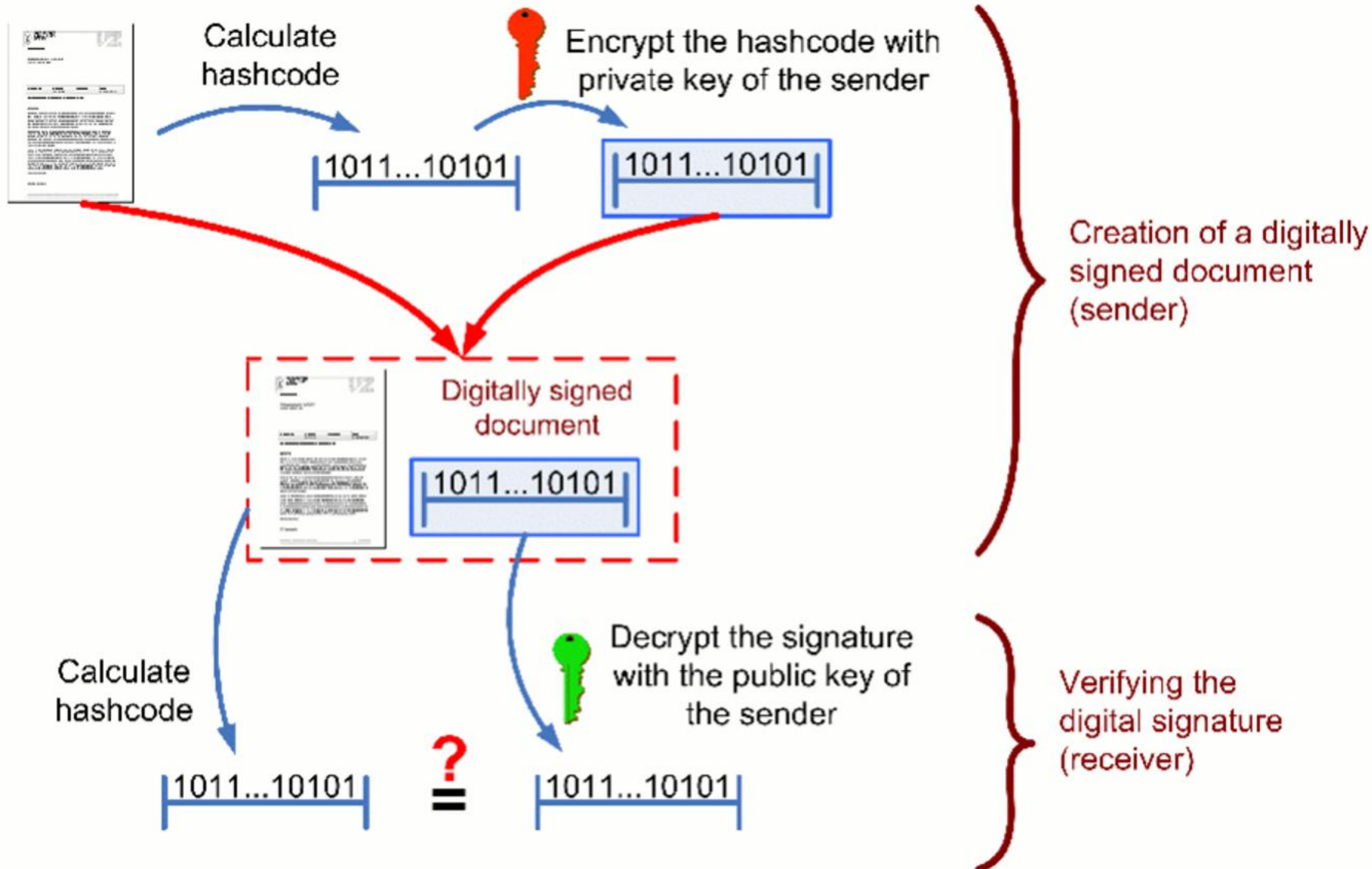
Security Guarantee: Only the person with the private key can create a signature that decrypts correctly with their public key.

6: Digital Signatures & Authentication (Cont'd)



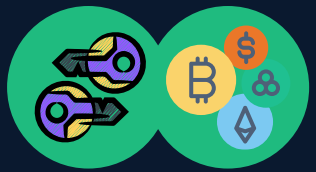
How Digital Signature Works?

Creating and verifying a digital signature



If the calculated hashcode does not match the result of the decrypted signature, either the document was changed after signing, or the signature was not generated with the private key of the alleged sender.

7: Public Key Infrastructure (PKI)



What is PKI?

PKI is not a single algorithm but a complete system of programs, protocols, and policies enabling secure data transmission over untrusted networks.

Components:

- Cryptographic algorithms (RSA, ECC, AES, SHA-256)
- Digital certificates
- Certification authorities
- Certificate repositories
- Key management policies

The Certificate Problem

Question: How do you know a public key really belongs to the person claiming it?

Scenario:

- You receive a message with a public key labeled "Alice"
- You encrypt a message using this public key
- How do you know this key really belongs to Alice?
- What if the attacker substituted their own key and is intercepting messages?

Answer: Digital Certificates and Certificate Authorities

Digital Certificates

A digital certificate is a digitally signed document binding:

- An entity's public key
- Their identity (name, email, organization)
- Certificate validity period
- Issuer's digital signature

Standard Format: X.509 (international standard for public key certificates)

Contents of an X.509 Certificate:

Version

Serial Number (unique identifier)

Signature Algorithm

Issuer (CA that signed it)

Valid From / Valid To (expiration dates)

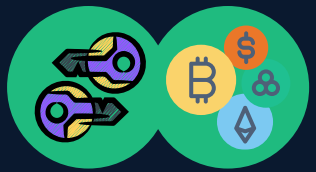
Subject (entity the certificate is for)

Subject Public Key Information

Extensions (usage constraints, SANs, etc.)

CA's Digital Signature

7: Public Key Infrastructure (PKI) (Cont'd)



Certification Authority (CA)

A Certification Authority (CA) is a trusted organization that:

1. Verifies applicant's identity (company registration, government ID, domain ownership)
2. Issues a digital certificate binding the public key to the identity
3. Signs the certificate with their private key
4. Maintains records of issued certificates
5. Manages certificate revocation

Trusted CAs:

- Globally trusted: Verisign, DigiCert, GoDaddy, Let's Encrypt
- Internal/Corporate: Company's own PKI infrastructure
- Government: National PKI providers

Trust Model: Users trust CA → Users accept certificates signed by CA

Certificate Chain

A chain of certificates back to the root CA proves trust.

Example Chain:

Leaf Certificate (your website)

- ← signed by Intermediate CA
- ← signed by another Intermediate CA
- ← signed by Root CA
(trusted by all browsers/systems)

Certificate Lifecycle

1. Issuance:

- User generates key pair
- Submits certificate signing request (CSR) with public key and identity info
- CA verifies identity
- CA signs the certificate with its private key
- Certificate issued and installed

2. Validity Period:

- Certificate valid from "Valid From" to "Valid To" dates
- Typical validity: 1 year (SSL/TLS), 3 years (corporate PKI)

3. Renewal:

- Before expiration, request a new certificate
- CA issues a new certificate with a new expiration date
- Keys remain the same if still secure
- Certificates must be renewed before expiration, or services stop

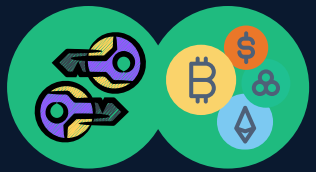
4. Revocation:

- Reason: Private key compromised, employee leaves, domain changes
- Difference from Suspension:
- Revocation: Permanent, cannot be undone
- Suspension: Temporary, can be lifted

Revocation Methods:

- CRL (Certificate Revocation List): List of revoked certificate serial numbers; must be checked regularly
- OCSP (Online Certificate Status Protocol): Real-time query to CA; faster than CRL
- OCSP Stapling: Server includes OCSP response with certificate; eliminates client-CA connection

7: Public Key Infrastructure (PKI) (Cont'd)



PKI Use Cases

1. HTTPS/SSL-TLS:

- Authenticates the web server
- Encrypts communication
- Every HTTPS connection is verified by PKI

2. Email Signing/Encryption:

- S/MIME certificates
- PGP web of trust
- Proves email authenticity

3. Code Signing:

- Software publishers sign executables
- Verifies software hasn't been modified
- Builds user trust

4. VPN/Wireless:

- Authentication for VPN servers
- Wi-Fi certificate-based authentication
- Enterprise access control

5. IoT Devices:

- Mutual authentication between devices
- Secure device-to-cloud communication
- Supply chain verification

HTTP Strict Transport Security (HSTS)

What it does: HTTP response header telling the browser "always load this site via HTTPS"

HSTS Header:

Strict-Transport-Security: `max-age=31536000; includeSubDomains; preload`

Parameters:

- **max-age:** Seconds to enforce HTTPS (31536000 = 1 year)
- **includeSubDomains:** Apply to all subdomains
- **preload:** Include in browser's hardcoded HSTS preload list

Security Benefit: Prevents downgrade attacks; the browser will never send HTTP requests even if the user types http://

Key Management Best Practices

Backing Up Keys:

- Critical for business continuity
- If the private key is lost and not backed up, the encrypted data becomes irretrievable
- CA is usually responsible for key backup
- Key recovery policy must exist

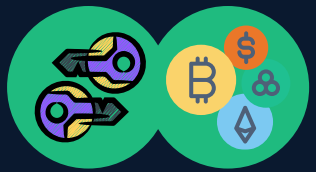
Key Storage:

- Private keys must be protected with strong passwords
- Use hardware security modules (HSMs) for high-value keys
- Never store private keys in plain text
- Separate keys by security level (encryption vs. signing)

Key Rotation:

- Regularly replace keys before compromise
- Schedule: 1-3 years for most certificates
- Establish rotation policy before needed

8: Cryptographic Attacks



Attack Classifications

Passive Attacks: Observe and capture data without modification

- Sniffing (packet capture)
- Traffic analysis
- Example: Network monitoring via sniffer tools (Wireshark, tcpdump)

Active Attacks: Attempt to modify data or determine the encryption key

- Brute force
- Man-in-the-middle
- Replay attacks
- Injection attacks

Brute Force Attack

Method: Try every possible key until the plaintext is revealed.

Characteristics:

- No mathematical insight required
- Guaranteed to work eventually
- Computational cost increases exponentially with key size
- Feasible only with small keys or insufficient processing time

Practical Timeline (with modern hardware):

- 40-bit key: Seconds to minutes
- 56-bit key (DES): Minutes to hours
- 128-bit key: Billions of years
- 256-bit key: Unfeasible within the universe's lifetime

Countermeasures:

- Use a minimum 128-bit key for symmetric (256-bit recommended)
- Use 2048-4096 bit keys for RSA
- Implement exponential backoff on password attempts
- Account lockout after failed attempts
- Use key stretching (PBKDF2, bcrypt, Argon2)

Dictionary Attack

Method: Use a pre-computed dictionary of common words and passwords.

Why Effective:

- Users choose predictable passwords (names, common words, dates)
- A dictionary contains millions of common passwords
- Very fast (millions of attempts per second)
- Successful against 50-90% of user passwords

Attack Timeline:

- Common 8-character password: < 1 hour
- Weak password database: Minutes to hours
- Strong/random passwords: Fails entirely

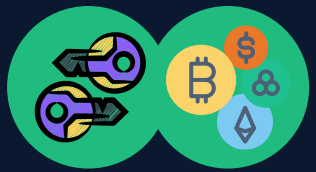
Tools:

- John the Ripper: Flexible, customizable
- Hashcat: GPU-accelerated, extremely fast
- Hydra: Network service authentication
- Cain & Abel: GUI-based Windows tool

Countermeasures:

- Require strong passwords (12+ characters, mixed case, numbers, symbols)
- Implement rate limiting on login attempts
- Use salted password hashing (prevents rainbow table attacks)
- Require MFA (even with a weak password, the account is protected)
- Educate users about password strength

8: Cryptographic Attacks (Cont'd)



Rainbow Table Attack

What is a Rainbow Table? Pre-computed hash-to-password mappings for millions of passwords.

Why Effective:

- Eliminates computation time during an attack
- Trade storage for speed
- One lookup finds the password instantly
- No computation needed

Example:

Hash: 5d41402abc4b2a76b9719d911017c592

Lookup in rainbow table → "hello"

Protection: Salting

A salt is random data added to a password before hashing.

Salted Hash:

```
hash = SHA-256("correct horse battery staple" + "n3k8@v9!")
```

Benefits:

- Same password produces different hashes with different salts
- Rainbow tables become useless (need one per salt)
- Forces the attacker back to slow computation
- Industry standard: 16+ byte random salt

Tools for Creating Rainbow Tables:

- Ophcrack (Windows password hashes)
- RainbowCrack (various hash types)
- Table lookup databases (available online)

Birthday Attack

Birthday Paradox: With 23 random people, 51% chance two share a birthday (not 50% as intuition suggests).

$$\frac{365}{365} \times \frac{364}{365} \times \frac{363}{365} \times \frac{363}{365} \times \frac{361}{365} \times \frac{360}{365} \times \dots \times \frac{343}{365} = 49\%$$

Applied to Hashing: Finding two documents with the same hash requires $\sim\sqrt{2^n}$ attempts, where n = hash size in bits.

Example - SHA-1 (160-bit):

- Theoretical security: 2^{160} attempts to find a specific match
- Birthday attack security: 2^{80} attempts to find any collision
- $2^{80} = \sim 10^{24}$ operations (feasible with distributed computing)

Impact:

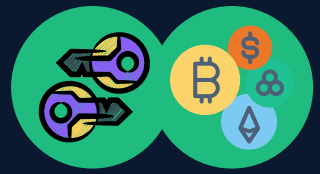
- MD5 (128-bit): Birthday birthday attack = $2^{64} = 18$ quintillion (2004 - practical)
- SHA-1 (160-bit): Birthday attack = $2^{80} = 1.2$ septillion (2005 - practical with effort)
- SHA-256 (256-bit): Birthday attack = 2^{128} = still unfeasible

Real-World Example: Researchers found two different PDF files with the same MD5 hash (2008), compromising digital signatures.

Prevention:

- Use hash algorithms with 256+ bit output (SHA-256, SHA-3)

Key Takeaways For Pentesters



- Cryptography secures data using mathematical techniques.
- Encryption evolved from simple ciphers to advanced algorithms.
- Classical ciphers are insecure and easily broken.
- Cryptography is essential for modern cybersecurity.

Thank You

ANY QUESTIONS?

